Hoja de Referencia: Sentencias y Anotaciones para Pruebas en Java

Guía Rápida de JUnit 5, Mockito y Spring Boot Test (Slices, Mocks, Aserciones y Buenas Prácticas)

1. JUNIT 5 - ANOTACIONES Y CICLO DE VIDA

Anotación / Sentencia	Descripción y Propósito	Ejemplo de Uso
`@Test`	Marca un método como una prueba unitaria ejecutable.	`@Test void validarCorreo() { ... }`
`@BeforeEach`	Se ejecuta antes de cada método de prueba en la clase.	`@BeforeEach void setUp() { ... }`
`@AfterEach`	Se ejecuta inmediatamente después de cada prueba.	`@AfterEach void tearDown() { ... }`
`@BeforeAll`	Se ejecuta una vez antes de todas las pruebas (método static).	`@BeforeAll static void init() { ... }`
`@ExtendWith`	Habilita extensiones de terceros (ej. MockitoExtension.class).	`@ExtendWith(MockitoExtension.class)`
`@DisplayName`	Asigna un nombre descriptivo personalizado a la prueba.	`@DisplayName("Validar correo")`
`@Disabled`	Deshabilita la prueba o clase para omitir su ejecución.	`@Disabled("En refactorización")`

2. SENTENCIAS DE ASERCIÓN (JUNIT 5 & ASSERTJ)

Sentencia de Aserción	Origen	Evaluación / Propósito
`assertEquals(esperado, actual)`	JUnit 5	Verifica igualdad entre el valor esperado y el retornado.
`assertTrue(condición)`	JUnit 5	Asegura que la expresión booleana evaluada sea true.
`assertFalse(condición)`	JUnit 5	Asegura que la expresión booleana evaluada sea false.
`assertNotNull(objeto)`	JUnit 5	Verifica que el objeto o resultado no sea nulo.
`assertThrows(Class, executable)`	JUnit 5	Comprueba que se lance la excepción esperada durante la ejecución.
`assertThat(actual).isEqualTo(esp)`	AssertJ	Aserción fluida y legible para comprobar igualdad.
`assertThat(actual).isNotNull()`	AssertJ	Aserción fluida para verificar que un objeto no sea nulo.

3. MOCKITO - CREACIÓN E INYECCIÓN DE OBJETOS SIMULADOS

Anotación / Objeto	Comportamiento y Uso Recomendado
`@Mock`	Crea un objeto simulado (dummy). Sus métodos retornan valores por defecto (null, 0, false) hasta ser configurados (stubbed). Aísla colaboradores externos.
`@InjectMocks`	Instancia la clase bajo prueba (System Under Test / SUT) e inyecta automáticamente los campos anotados con @Mock o @Spy.
`@Spy`	Crea un simulador parcial sobre un objeto real. Ejecuta los métodos reales salvo aquellos configurados explícitamente.
`@Captor`	Declara un ArgumentCaptor para interceptar y verificar los parámetros pasados a un mock durante la prueba.

Código de Ejemplo: Prueba Unitaria Aislada con JUnit 5 + Mockito

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {
    @Mock private UserRepository userRepository; // Colaborador simulado
    @InjectMocks private UserService userService; // SUT con mocks inyectados
    @Captor private ArgumentCaptor<User> userCaptor;

    @Test
    void testCrearUsuario() {
        User user = new User(1L, "Ana", "ana@ejemplo.com");
        when(userRepository.save(any(User.class))).thenReturn(user);

        User resultado = userService.crearUsuario("Ana", "ana@ejemplo.com");

        assertNotNull(resultado);
        verify(userRepository).save(userCaptor.capture());
        assertEquals("ana@ejemplo.com", userCaptor.getValue().getEmail());
    }
}
```

4. MOCKITO - STUBBING, VERIFICACIÓN Y COINCIDENCIA DE ARGUMENTOS

Sentencia de Mockito	Descripción y Caso de Uso
`when(mock.metodo()).thenReturn(valor)`	Configura un método no-void para devolver un valor predeterminado al ser invocado.
`when(mock.metodo()).thenThrow(Excepción.class)`	Configura el mock para lanzar una excepción al ejecutarse el método.
`doReturn(valor).when(spy).metodo()`	**Obligatorio en @Spy:** Evita ejecutar el método real del objeto espía durante la configuración.
`doNothing().when(mock).metodoVoid()`	Configura métodos de retorno void para ignorar la ejecución silenciosamente.
`verify(mock, times(n)).metodo()`	Verifica que el método simulado se haya ejecutado exactamente 'n' veces.
`verifyNoInteractions(mock)`	Garantiza que nunca se haya llamado a ningún método del objeto simulado.
`verifyNoMoreInteractions(mock)`	Asegura que no existan interacciones adicionales no verificadas en el mock.
`any(), eq(val), anyString(), argThat(pred)`	ArgumentMatchers para especificar coincidencias flexibles o predicados personalizados.

5. PRUEBAS EN SPRING BOOT - ANOTACIONES, SLICES Y HERRAMIENTAS

Anotación / Componente	Alcance / Propósito	Uso / Herramienta Asociada
`@SpringBootTest`	Carga el ApplicationContext completo de Spring Boot. Para pruebas de integración de extremo a extremo.	`webEnvironment = RANDOM_PORT, TestRestTemplate, RestTestClient`
`@WebMvcTest(Controller.class)`	Slice test especializado únicamente en la capa Web (Controllers, Mappings, Filtros REST).	`MockMvc, @MockBean / @MockitoBean para simular servicios`
`@DataJpaTest`	Slice test especializado en la capa de datos (JPA Repositories, Entities). Activa base de datos embebida.	`TestEntityManager, H2 Database, @Autowired Repositories`
`@MockBean / @MockitoBean`	Reemplaza o añade un mock de Mockito dentro del Spring TestContext durante la prueba.	`Sustituye beans reales en @SpringBootTest y @WebMvcTest`
`@SpyBean`	Envuelve un bean existente del Spring TestContext en un espía de Mockito.	`Mantiene funcionalidad real permitiendo stubbing parcial`
`@LocalServerPort`	Inyecta el puerto dinámico asignado al servidor web embebido en pruebas con puerto aleatorio.	`private int port;`

Código de Ejemplo: Pruebas Web Sliced (@WebMvcTest) e Integración (@SpringBootTest)

```
// 1. Slice Test Web con @WebMvcTest y MockMvc
@WebMvcTest(TaskController.class)
class TaskControllerTest {
    @Autowired private MockMvc mockMvc;
    @MockBean private TaskService taskService; // Inyecta mock en el contexto de Spring

    @Test
    void testObtenerTarea() throws Exception {
        when(taskService.getTaskById(1L)).thenReturn(new TaskDto(1L, "Prueba"));
        mockMvc.perform(get("/api/tasks/1"))
               .andExpect(status().isOk())
               .andExpect(jsonPath("$.title").value("Prueba"));
    }
}

// 2. Prueba de Integración Completa con @SpringBootTest
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class TaskIntegrationTest {
    @LocalServerPort private int port;
    @Autowired private TestRestTemplate restTemplate;

    @Test
    void testFlujoCompleto() {
        ResponseEntity<TaskDto> response = restTemplate.getForEntity("http://localhost:" + port + "/api/tasks/1", TaskDto.class);
        assertEquals(HttpStatus.OK, response.getStatusCode());
    }
}
```

6. ERRORES FRECUENTES Y BUENAS PRÁCTICAS DE DESARROLLO

Error / Mal Hábitos	Explicación y Solución Recomendada
`Confundir @Mock y @MockBean`	**@Mock** es para unit tests sin Spring Context. **@MockBean / @MockitoBean** reemplaza un bean dentro del contexto de Spring en pruebas integradas.
`Uso excesivo de @SpringBootTest`	Iniciar todo el contexto en cada prueba degrada el rendimiento del build. Use unit tests para reglas de negocio y test slices (@WebMvcTest, @DataJpaTest) para capas específicas.
`Usar when() en objetos @Spy`	Configurar un espía con when(spy.metodo()).thenReturn() ejecuta el método real durante la configuración. Use siempre doReturn(val).when(spy).metodo().
`Mezclar JUnit 4 y JUnit 5`	Mezclar paquetes de JUnit 4 (org.junit) y JUnit 5 (org.junit.jupiter) provoca la omisión silenciosa de pruebas. Mantenga únicamente dependencias y anotaciones de JUnit 5.